

Relatório Técnico

Agente Comparador de Apólices D&O (ACADeO)

1. Resumo executivo

O ACADeO é um MVP de aplicação de Inteligência Artificial Generativa para apoio à análise e comparação de apólices de seguro D&O.

A proposta é transformar documentos extensos e não estruturados em informações organizadas, permitindo que o usuário faça upload de uma apólice, obtenha uma extração automática e compare duas apólices previamente processadas.

O projeto prioriza uma arquitetura simples e demonstrável, adequada ao escopo de prova de conceito.

2. Problema

Apólices de seguro D&O são documentos extensos, com informações distribuídas entre diferentes seções e cláusulas.

A comparação manual exige localizar informações, interpretar redações e identificar diferenças entre documentos.

O ACADeO busca reduzir o trabalho mecânico de localização e organização das informações, mantendo a leitura humana como etapa necessária quando há diferenças textuais ou cláusulas que exigem interpretação.

3. Objetivo

Construir uma solução capaz de:

1. receber uma apólice em PDF ou imagem;
2. extrair automaticamente informações relevantes;
3. estruturar os dados;
4. armazenar as análises;
5. consultar documentos processados;
6. comparar pelo menos duas apólices;
7. apresentar as principais diferenças em uma interface gráfica.

4. Escopo do MVP

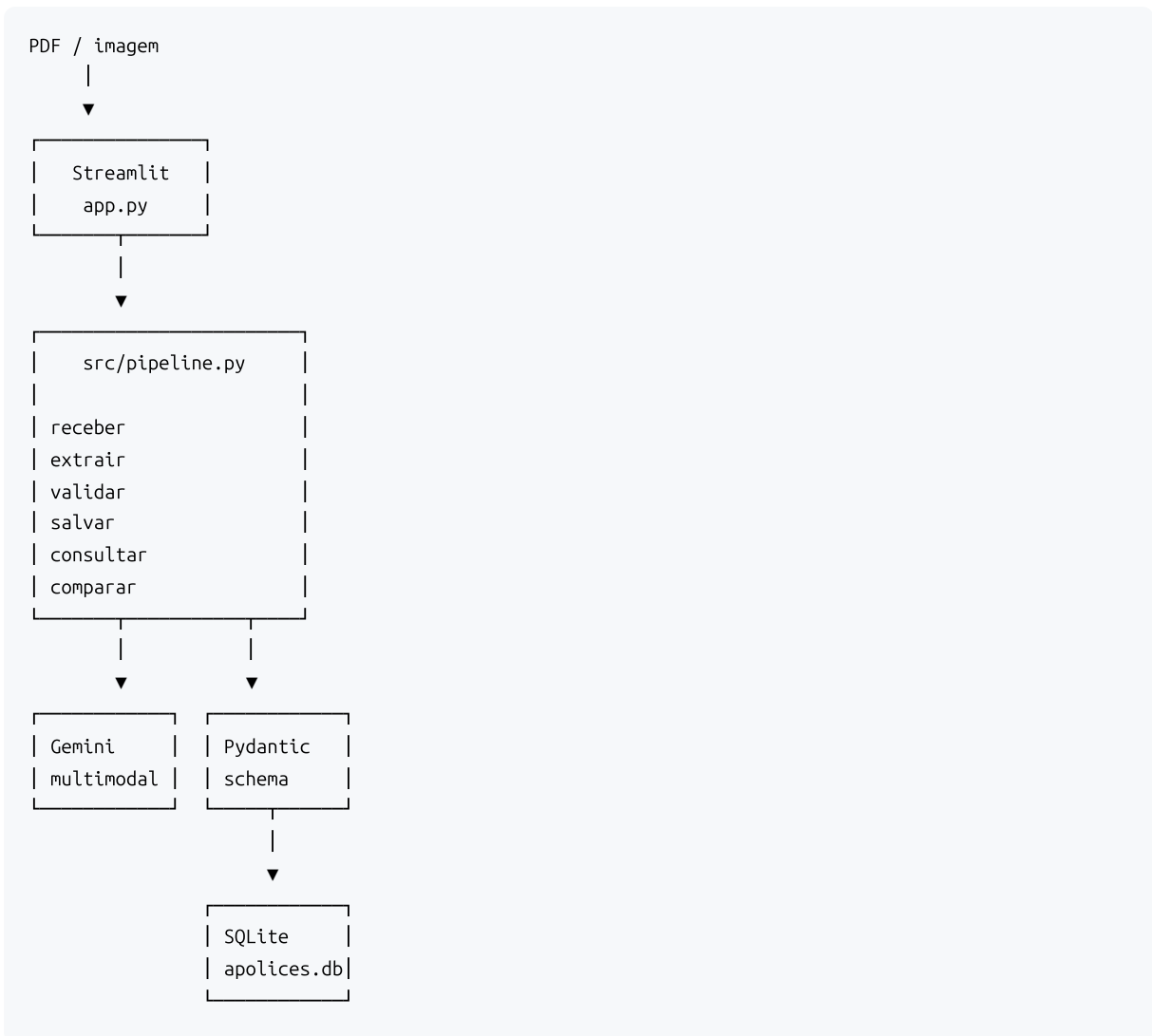
O MVP contempla:

- upload de PDF e imagens;
- extração multimodal com Gemini;
- resposta em JSON estruturado;
- validação com Pydantic;
- persistência em SQLite;
- consulta das apólices processadas;
- comparação de duas apólices;
- identificação de diferenças de campos;
- identificação de exclusões presentes somente em uma das apólices;
- interface Streamlit.

O projeto não pretende ser um sistema comercial de subscrição, regulação ou aconselhamento jurídico.

5. Arquitetura da solução

Relatório Técnico



A separação entre interface e pipeline permite que a mesma lógica seja utilizada pela interface Streamlit e pela execução via linha de comando.

6. Tecnologias utilizadas

Uso de Inteligência Artificial no Desenvolvimento

Durante o desenvolvimento do ACADeO, ferramentas de Inteligência Artificial Generativa foram utilizadas como apoio ao processo de programação e documentação.

A IA foi utilizada para auxiliar na elaboração e revisão de código Python, investigação de erros, organização da estrutura do projeto, discussão de decisões arquiteturais e produção da documentação. Os resultados gerados foram revisados e testados no ambiente do projeto antes de serem incorporados à solução.

No produto desenvolvido, a Inteligência Artificial também constitui um componente central da arquitetura. O ACADeO utiliza o modelo Gemini para realizar a análise multimodal dos documentos enviados e extrair informações relevantes das apólices de D&O. A resposta do modelo é solicitada em formato JSON estruturado e posteriormente validada por modelos Pydantic.

O uso da IA, portanto, ocorre em duas dimensões distintas: como ferramenta de apoio ao desenvolvimento e como tecnologia funcional incorporada ao produto.

A utilização de IA generativa não elimina a necessidade de validação humana. No contexto deste MVP, os resultados da extração devem ser considerados como apoio à análise, e não como substituição da leitura ou avaliação profissional das cláusulas das apólices.

Relatório Técnico

- Python: Linguagem principal da solução.
- Streamlit: Utilizado na construção da interface para upload, consulta e comparação.
- Google Gemini / Google GenAI: Utilizado para a extração multimodal dos documentos.
- Pydantic: Utilizado para definir o modelo estruturado da apólice e validar a resposta JSON.
- SQLite: Utilizado para persistência local das análises.
- python-dotenv: Utilizado para carregar a chave da API a partir do arquivo `.env`.

7. Processo de ingestão

A função `receber()` verifica:

- existência do arquivo;
- extensão aceita;
- arquivo não vazio;
- limite máximo de 20 MB.

As extensões aceitas no pipeline são PDF, PNG, JPG, JPEG e WEBP.

8. Extração com IA Generativa

A função `extrair()` utiliza o SDK do Google GenAI.

O documento é enviado diretamente ao Gemini junto com um prompt especializado.

O prompt determina que o modelo:

- extraia somente informações explicitamente presentes;
- ignore sumário, glossário e assinaturas;
- priorize quadro-resumo, frontispício, condições gerais e cláusulas específicas;
- não invente informações;
- preserve informações textuais relevantes;
- identifique exclusões explicitamente presentes;
- forneça evidências com página e trecho;
- retorne somente o JSON esperado.

A aplicação utiliza resposta JSON estruturada por schema e temperatura baixa.

9. Estrutura dos dados

O modelo `ApoLiceDO` organiza informações como:

- seguradora;
- tomador;
- processo SUSEP;
- tipo de documento;
- início e fim da vigência;
- base de reclamações;
- LMI;
- limite agregado;
- prêmio;
- POS/franquia;
- coberturas A, B e C;
- custos de defesa;
- prazo complementar;
- prazo suplementar;
- retroatividade;
- territorialidade;
- exclusões;

- evidências;
- confiabilidade;
- alertas.

O modelo `Evidencia` registra:

- campo;
- página;
- trecho literal.

10. Validação e confiabilidade

Após a resposta do Gemini, o JSON é validado por Pydantic.

O sistema também calcula um indicador heurístico de confiabilidade com base na quantidade de campos relevantes preenchidos e na presença de evidências.

Os níveis utilizados são:

- alta;
- média;
- baixa.

Esse indicador não representa uma probabilidade estatística de acerto.

Quando a extração apresenta problemas, o sistema registra alertas, como recomendação de revisão humana.

11. Persistência

Os dados são armazenados em SQLite.

A tabela `apolices` contém:

Campo	Finalidade
<code>id</code>	Identificador interno
<code>arquivo</code>	Nome do documento
<code>processado_em</code>	Data/hora do processamento
<code>seguradora</code>	Seguradora identificada
<code>tipo_documento</code>	Tipo do documento
<code>confiabilidade</code>	Indicador heurístico
<code>lmi_valor</code>	LMI numérico quando disponível
<code>json_completo</code>	Resultado estruturado completo

A escolha pelo SQLite reduz a complexidade de infraestrutura e é suficiente para o objetivo demonstrativo do MVP.

12. Comparação

A função `comparar()` organiza a análise em duas categorias principais.

Valores numéricos

Para LMI e limite agregado:

- verifica ausência de valores;
- identifica igualdade;

Relatório Técnico

- calcula a diferença percentual quando os dois valores estão disponíveis.

Informações textuais

Para coberturas, base de reclamações, POS/franquia, prazos, retroatividade e territorialidade:

- identifica ausência;
- identifica igualdade;
- sinaliza textos diferentes para análise humana.

O sistema evita concluir automaticamente que uma cláusula textual é melhor ou pior apenas porque sua redação é diferente.

Exclusões

As exclusões são comparadas por diferença literal entre as listas extraídas.

A interface apresenta:

- exclusões identificadas somente na Apólice A;
- exclusões identificadas somente na Apólice B.

Isso não significa que uma apólice necessariamente ofereça cobertura sobre um item ausente da lista da outra. A interpretação contratual continua dependendo da leitura da cláusula.

13. Interface

A interface possui três áreas principais.

Upload e Análise

Permite:

- selecionar PDF/imagem;
- analisar documento;
- detectar arquivo já processado;
- cancelar ou confirmar reprocessamento;
- visualizar dados extraídos;
- visualizar indicador de confiabilidade e alertas.

Apólices Salvas

Permite consultar os registros armazenados no banco.

Comparar Apólices

Permite:

- selecionar duas apólices;
- visualizar os documentos escolhidos;
- comparar campos;
- visualizar indicadores de igualdade/diferença;
- consultar exclusões identificadas somente em cada apólice.

14. Rastreabilidade

O pipeline possui estrutura de evidências associada aos campos extraídos.

Cada evidência pode registrar página e trecho literal do documento.

Essa estrutura permite, em uma evolução futura, apresentar na interface a relação:

Relatório Técnico

```
Campo extraído
↓
Página do documento
↓
Trecho de evidência
```

No MVP, a rastreabilidade existe no dado estruturado, mas não foi transformada em uma tela específica de evidências.

15. Tratamento de erros

O pipeline trata situações como:

- arquivo inexistente;
- extensão não suportada;
- arquivo vazio;
- arquivo acima de 20 MB;
- ausência da chave Gemini;
- indisponibilidade temporária do serviço;
- esgotamento de cota;
- resposta incompatível com o JSON esperado;
- tentativa de consulta de apólice inexistente.

Em erros temporários do Gemini, o pipeline realiza novas tentativas antes de interromper o processamento.

16. Decisões arquiteturais

Arquitetura simples

A solução não utiliza múltiplos agentes autônomos. Para o escopo do MVP, um pipeline único é suficiente para demonstrar a utilização de GenAI, estruturação, persistência e comparação.

Gemini multimodal

Foi escolhido porque o documento é a entrada principal do sistema e pode conter informações cuja localização e interpretação dependem do conteúdo visual/textual da apólice.

Pydantic

Foi escolhido para transformar a resposta generativa em uma estrutura de dados explícita e validável.

SQLite

Foi escolhido pela baixa complexidade operacional e adequação ao protótipo.

Streamlit

Foi escolhido para reduzir o esforço de desenvolvimento da camada de apresentação e permitir uma demonstração funcional.

Vector database

O diretório `vectorbase` foi criado como ponto de extensão arquitetural. A versão entregue não utiliza banco vetorial, pois o MVP atual não depende de busca semântica para realizar a comparação.

17. Limitações

As principais limitações identificadas são:

Relatório Técnico

1. **Variabilidade da IA:** resultados podem variar entre execuções.
2. **Dependência da qualidade documental:** PDFs/imagens pouco legíveis podem reduzir a qualidade da extração.
3. **Comparação textual:** textos diferentes não são necessariamente coberturas diferentes.
4. **Exclusões:** a comparação atual é literal, não semântica.
5. **Confiabilidade:** o indicador é heurístico.
6. **Revisão humana:** diferenças contratuais relevantes continuam exigindo leitura especializada.
7. **Escalabilidade:** SQLite e execução local são escolhas de MVP.
8. **Segurança:** não há autenticação, autorização ou infraestrutura de produção.
9. **Integrações:** não há integração com seguradoras ou sistemas externos.
10. **Vectorbase:** não está operacional nesta versão.

Essas limitações são compatíveis com o caráter de prova de conceito do projeto.

18. Evolução futura

Como evolução, o projeto pode incorporar:

- RAG;
- banco vetorial;
- comparação semântica;
- classificação de cláusulas;
- exibição das evidências na interface;
- geração automática de relatório comparativo;
- API;
- banco de dados externo;
- autenticação;
- controle de acesso;
- monitoramento;
- testes automatizados de qualidade da extração.

Essas funcionalidades não são necessárias para o funcionamento do MVP entregue.

19. Como executar

Na raiz do projeto:

```
python -m venv .envv
source .envv/bin/activate
pip install -r requirements.txt
```

Configurar `.env` :

```
GOOGLE_API_KEY=sua_chave_aqui
```

Executar:

```
streamlit run app.py
```

20. Demonstração sugerida

Para a apresentação:

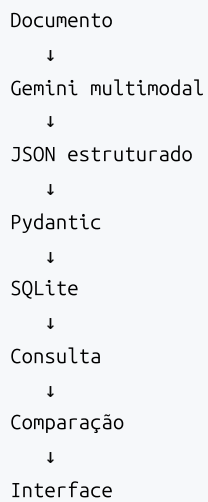
1. abrir a aplicação;
2. fazer upload de uma apólice;
3. mostrar a extração automática;

Relatório Técnico

4. mostrar seguradora, tipo, LMI e demais campos;
5. carregar uma segunda apólice;
6. abrir Comparar Apólices ;
7. selecionar as duas apólices;
8. executar a comparação;
9. mostrar diferenças numéricas e textuais;
10. mostrar exclusões identificadas somente em cada apólice;
11. explicar que diferenças textuais exigem análise humana.

21. Conclusão

O ACADeO demonstra um fluxo completo de aplicação de IA generativa a documentos de seguro D&O:



O principal resultado do MVP é a integração funcional entre modelo generativo, estruturação de dados, persistência e interface de comparação.

A arquitetura foi mantida deliberadamente simples para facilitar compreensão, demonstração e evolução posterior.